Parallel Text Search

Zwick, Patrick
patrick.zwick@htwg-konstanz.de

Döbele, Max
max.doebele@htwg-konstanz.de

06 February 2026

Abstract

1 Introduction

2 Benchmarking

For testing, we use a collection of up to 98 books from Project Gutenberg, which we concatenate into a single text containing over 82 million characters. We perform queries using, at most, the 100 most common English words. Since most of the 100 most common English words are very short, we additionally use a second query set consisting of randomly selected longer English words with lengths between 7 and 10 characters.

Both the number of books and the number of queries can be varied to evaluate performance under different conditions.

Performance benchmarks were conducted on a system equipped with an Intel Core i5-14600K CPU (featuring 6 P-cores at 3.5 GHz and 8 E-cores at 2.6 GHz) and an AMD Radeon RX 7800 XT GPU.

3 Sequential Implementations

We first implemented several purely sequential versions of the algorithm. The objective was not to achieve peak performance through highly optimized or specialized techniques from computer graphics or numerical computing, but rather to identify opportunities for exploiting data-level and task-level parallelism across different hardware targets. These sequential implementations serve as a baseline for analyzing the structure of the computation and for identifying points where the problem can later be decomposed into independent tasks that are well suited to heterogeneous hardware architectures.

3.1 Reference Implementation

To verify correctness, we use a simple baseline implementation, denoted `std`, which relies on `std::string::find()` for substring matching (see Listing 1).

3.2 Candidate Selection and Validation

The first implementation is divided into two distinct phases:

1. Identification of candidate positions for potential occurrences based on a simple filtering criterion. A position is considered a candidate if three characters match: the first character of the query, the middle character, and the last character.

2. Verification of each candidate position to determine whether it constitutes a valid occurrence.

The initial approach, denoted `candidate_v1`, performs candidate identification followed by validation separately for each query (see Listing 2). The vector is cleared after each query and reused for the next, avoiding unnecessary allocations.

```cpp
std::vector<std::vector<size_t>>
find_std(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        auto pos = text.find(query);
        while (pos != std::string::npos) {
            indices[i].push_back(pos);
            pos = text.find(query, pos + 1);
        }
    }
    return indices;
}
```

Listing 1: `std` reference implementation (`std_text_search.cpp`).

```cpp
void find_candidates(const size_t text_length, std::vector<size_t> &results,
                     const std::string &text, const std::string &query) {
    const auto query_length = query.length();
    const auto mid = query_length >> 1;
    const auto end = query_length - 1;
    for (size_t i = 0; i < text_length - query_length; ++i) {
        if (text[i] == query[0] && text[i + mid] == query[mid] &&
            text[i + end] == query[end]) {
            results.push_back(i);
        }
    }
}

bool test_candidate(const size_t index, const std::string &text,
                    const std::string &query) {
    return std::memcmp(text.data() + index, query.data(), query.size()) == 0;
}

std::vector<std::vector<size_t>>
find_candidate_v1(const std::string &text,
                  const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const auto text_length = text.length();
    std::vector<size_t> results;
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        find_candidates(text_length, results, text, query);
        for (const auto &result : results) {
            if (test_candidate(result, text, query)) {
                indices[i].push_back(result);
            }
        }
        std::memset(results.data(), 0, results.size() * sizeof(size_t));
    }

    return indices;
}
```

Listing 2: `candidate_v1` implementation (`candidate_v1_text_search.cpp`).

3.2.1 Array Pre-Allocation Instead of Dynamic Vector Growth

Since a `std::vector` may need to repeatedly grow its underlying storage and copy existing elements when its capacity is exceeded if no default capacity is specified, this can introduce a significant performance bottleneck, at least for the first query. To avoid this overhead, the `candidate_v2` implementation instead pre-allocates an array with a size equal to the length of the text. In the worst case, every position in the text could be a valid candidate, so this allocation is sufficient to store all candidates.

3.2.2 Using a Bitmask

To improve CPU cache utilization and reduce the memory footprint, the `candidate_v3` implementation represents the candidate buffer as a *bitmask*, where each bit set to 1 denotes a valid candidate position. The bitmask is stored as an array of unsigned 64-bit integers, allowing 64 candidates to be packed into a single word.

Let m denote the length of the text. To represent candidate positions for a query of any length, we require

$$h = \left\lceil \frac{m + 63}{64} \right\rceil$$

64-bit words in the bitmask.

For a candidate at position k, the word index w of the corresponding 64-bit word B_w in the bitmask is obtained by dividing k by 64 which can be efficiently computed by a right shift of 6 bits:

$$w = \frac{k}{64} = k \gg 6.$$

The bit position b within the word B_w is given by taking k modulo 64, which can be efficiently computed by a bitwise AND with 63:

$$b = k \bmod 64 = k \,\&\, 63.$$

To set this bit, we construct a temporary value with only the target bit active by shifting the integer constant 1 to the left by the bit position b, and then update the mask word using the bitwise OR:

$$B_w \leftarrow B_w \mid (1 \ll b).$$

To reconstruct candidate positions from the bitmask, we iterate over all 64-bit words and extract the indices of the set bits.

For a given word index w, let B_w again denote the corresponding 64-bit word. If $B_w = 0$, the word contains no valid candidates. Otherwise, we repeatedly extract the least significant set bit until B_w becomes zero.

The position of this bit within the word is obtained using the number of trailing zeros:

$$b = \mathrm{ctz}(B_w),$$

where ctz denotes the count of trailing zero bits.

The corresponding global candidate index is then

$$k = 64w + b.$$

After processing a candidate, the lowest set bit is cleared using the operation

$$B_w \leftarrow B_w \,\&\, (B_w - 1),$$

which removes the extracted bit and guarantees termination after all set bits have been processed.

This approach ensures that candidate extraction runs in time proportional to the number of valid candidates rather than the size of the bitmask.

3.2.3 Using a Bitmask for All Queries Together

Instead of allocating a separate bitmask for each query and clearing it after processing, the `candidate_v4` implementation uses single bitmask large enough to store candidate positions for all queries simultaneously. By restructuring the search to iterate primarily over the text followed by the queries, the implementation achieves a more cache-friendly access pattern while populating this unified bitmask.

For n queries, the total bitmask size becomes

$$h_{\text{total}} = n \cdot h,$$

where h is the number of 64-bit words needed for a single query.

To access the word corresponding to a candidate at position k for query q, we first compute the offset for that query:

$$l = q \cdot h.$$

The global word index in the combined bitmask is then

$$w_{\text{total}} = l + (k \gg 6),$$

where, as before, $k \gg 6$ gives the word index within the query's portion of the bitmask.

Reconstructing candidate positions proceeds in a similar manner. We iterate over all queries q, and within each query, over all words w from 0 to $h - 1$. For each word index, the offset-adjusted index is

$$w_{\text{total}} = l + w.$$

Candidate positions are then extracted from the set bits of $B_{w_{\text{total}}}$ using the same approach described previously.

3.3 Rabin-Karp Rolling-Hash

As one of several approaches, we also evaluate a Rabin–Karp–based rolling-hash in the `hash_v1` implementation (see Listing 3), which is widely used in text search due to its favorable average-case time complexity [1].

Let the query be a string of length M, with characters $q_0, q_1, \ldots, q_{M-1}$. Its hash is computed as

$$h(q) = \sum_{i=0}^{M-1} q_i \, B^{M-1-i},$$

where B is a prime number used as the base, and q_i is the numeric value of the i-th character.

Similarly, for a substring (window) of length M in the text starting at position j, with characters $t_j, t_{j+1}, \ldots, t_{j+M-1}$, the hash is

$$h_j = \sum_{i=0}^{M-1} t_{j+i} \, B^{M-1-i}.$$

To avoid recomputing h_j from scratch for each window, we apply the Rabin-Karp rolling hash formula:

$$h_{j+1} = \left(h_j - t_j \, B^{M-1}\right) B + t_{j+M}.$$

Whenever a window hash matches the query hash, a direct character-by-character comparison is performed to verify the match and eliminate potential hash collisions. This procedure is repeated independently for each query in the input set. All arithmetic is performed using 64-bit unsigned integers, implicitly modulo 2^{64}.

```cpp
#define PRIME 1000003

uint64_t compute_power(const size_t length) {
    uint64_t p = 1;
    for (size_t i = 1; i < length; ++i) {
        p *= PRIME;
    }
    return p;
}

uint64_t hash_string(const char *s, const size_t length) {
    uint64_t h = 0;
    for (size_t i = 0; i < length; ++i) {
        h = h * PRIME + static_cast<unsigned char>(s[i]);
    }
    return h;
}

std::vector<std::vector<size_t>>
find_hash_v1(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const size_t text_length = text.length();
    for (size_t i = 0; i < queries.size(); ++i) {
        const std::string &query = queries[i];
        const size_t query_length = query.length();
        if (query_length == 0 || query_length > text_length) {
            continue;
        }
        const uint64_t query_hash = hash_string(query.data(), query_length);
        const uint64_t power = compute_power(query_length);
        uint64_t window_hash = hash_string(text.data(), query_length);
        for (size_t j = 0; j <= text_length - query_length; ++j) {
            if (window_hash == query_hash &&
                std::memcmp(text.data() + j, query.data(), query_length) == 0) {
                indices[i].push_back(j);
            }
            if (j < text_length - query_length) {
                window_hash -= static_cast<unsigned char>(text[j]) * power;
                window_hash = window_hash * PRIME + static_cast<unsigned char>(
                                                    text[j + query_length]);
            }
        }
    }
    return indices;
}
```

Listing 3: `hash_v1` implementation (`hash_v1_text_search.cpp`).

3.4 Grouping Queries by Length

The first implementation `hash_v1` scans the entire text separately for each query. Given a large text that exceeds the CPU's L1 and L2 cache capacities, this approach creates a performance bottleneck, since the text must be reloaded from RAM for every query.

To improve memory bandwidth utilization and temporal cache locality, `hash_v2` groups all queries by their length (see Listing 4). For each group of queries of the same length, a `std::unordered_map` is used as a hash table, mapping each query hash to the indices of all queries sharing that hash. This allows every text window of the corresponding length to be checked simultaneously against all relevant queries in constant average time. When a hash match occurs, the substring is verified against the original query to eliminate false positives caused by hash collisions. By grouping queries and using a hash table, the text is scanned only once per query length, rather than once per individual query, greatly reducing memory traffic and runtime. The rolling hash itself is computed as in `hash_v1`, using a precomputed power of B to efficiently update the hash from one window to the next.

3.5 Evaluation

In Figure 1 and Figure 2, we observe that the `candidate_v1`, `candidate_v2`, and `hash_v1` variants consistently exhibit the lowest performance, independent of the number of books queried, the number of queries, or whether common or long words are used.

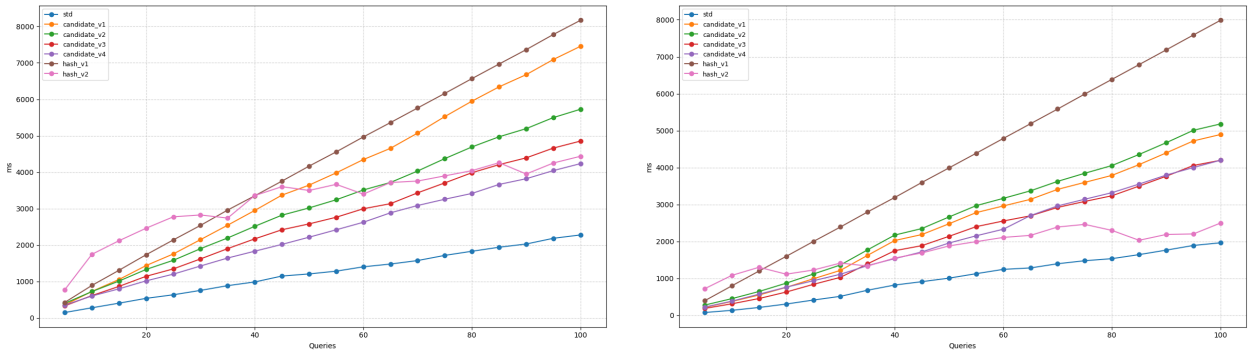
An exception occurs for low query counts, up to 35 common-word queries or 20 long-word queries, where `hash_v2` performs worse than the other variants, likely due to initialization overhead.

By contrast, the `std` variant consistently achieves the best performance across all test cases, which can be attributed to its implementation. In libstdc++ for example, `std::basic_string::find()` performs a search optimized with low-level memory operations, likely leveraging SIMD instructions for most architectures. Furthermore, the implementation differentiates between single-character searches and longer query strings, reducing overhead in common cases [2].

Our implementations, in contrast, do not employ length-dependent optimizations. All query strings are processed using a uniform search strategy, independent of their length. While this approach simplifies the code, incorporating query-length–specific optimizations similar to those in libstdc++ could further improve performance.

The bitmask-based approaches, `candidate_v3` and `candidate_v4`, show similar performance for long, less-common words. However, `candidate_v4` performs significantly better for common words.

This is due to different memory access patterns. In `candidate_v3`, the text is reloaded from main memory for every query, causing cache thrashing; the large text data constantly evicts the bitmask, which must be updated much more frequently for common words. In the `candidate_v4` approach, the text is scanned only once. This allows the bitmask to stay – at least partly – within the cache.



(a) Common words.

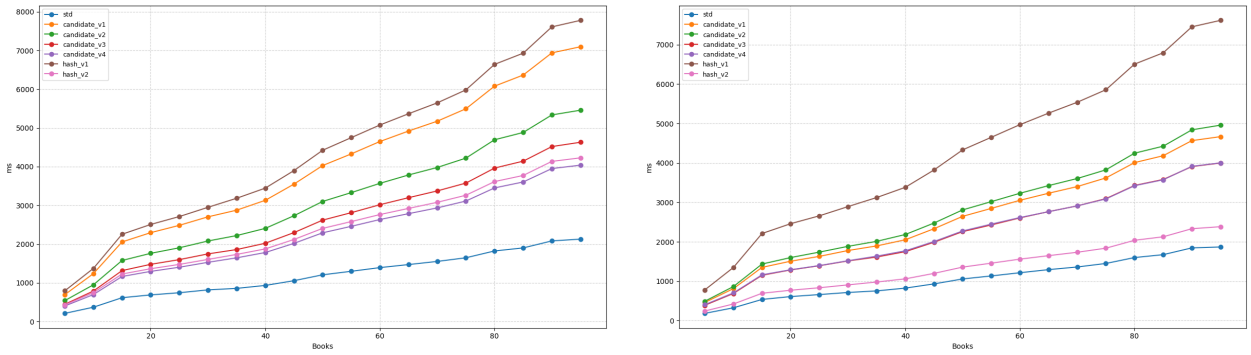
(b) Long words.

Figure 1: Runtime for sequential implementations for varying query counts (98 books).

```cpp
std::vector<std::vector<size_t>>
find_hash_v2(const std::string &text, const std::vector<std::string> &queries) {
    std::vector<std::vector<size_t>> indices(queries.size());
    const size_t text_length = text.length();
    std::unordered_map<size_t, std::vector<size_t>> length_groups;
    for (size_t i = 0; i < queries.size(); ++i) {
        length_groups[queries[i].length()].push_back(i);
    }
    for (auto const &[query_length, query_indices] : length_groups) {
        if (query_length == 0 || query_length > text_length) {
            continue;
        }
        std::unordered_map<uint64_t, std::vector<size_t>> hash_table;
        for (size_t q : query_indices) {
            uint64_t h = hash_string(queries[q].data(), query_length);
            hash_table[h].push_back(q);
        }
        const uint64_t power = compute_power(query_length);
        uint64_t window_hash = hash_string(text.data(), query_length);
        for (size_t j = 0; j <= text_length - query_length; ++j) {
            auto it = hash_table.find(window_hash);
            if (it != hash_table.end()) {
                for (const auto q : it->second) {
                    if (std::memcmp(text.data() + j, queries[q].data(),
                                    query_length) == 0) {
                        indices[q].push_back(j);
                    }
                }
            }
            if (j < text_length - query_length) {
                window_hash -= static_cast<unsigned char>(text[j]) * power;
                window_hash = window_hash * PRIME + static_cast<unsigned char>(
                                                        text[j + query_length]);
            }
        }
    }

    return indices;
}
```

Listing 4: `hash_v2` implementation (`hash_v2_text_search.cpp`).



(a) Common words.

(b) Long words.

Figure 2: Runtime for sequential implementations for varying book counts (100 queries).

4 Parallelization Across CPU Cores Using OpenMP

4.1 Candidate Selection and Validation

We use the two best bitmask-based approaches, `candidate_v3` and `candidate_v4`, in the implementations `candidate_openmp_v1` and `candidate_openmp_v2`, respectively. The queries are distributed across all CPU threads for both the selection and validation phases.

In the approach that creates a bitmask for each query, a separate bitmask is generated for every query, instead of creating a single bitmask once and reusing it for all queries. When using a single large bitmask for all queries, the bitwise OR must be replaced with its atomic counterpart to avoid race conditions. OpenMP provides the directive `#pragma omp atomic` for this purpose. This data dependency introduces a disadvantage compared to the per-query bitmask approach.

For the validation phase, no synchronization is required, since each query maintains its own vector within the overall result vector.

4.2 Rolling Hash

To parallelize the hash approach, we assign a group of queries of identical length to each thread. Since OpenMP requires a random access iterator or an integer index for the distribution of loops `#pragma omp parallel for`, a `std::unordered_map` cannot be used directly for parallelization. In the `hash_openmp_v1` implementation, we use a vector in which the query indexes are sorted by length. In addition, a second vector with a special block structure (LengthBlock) is created. This structure stores the respective word length, the start index within the sorted vector, and the number of words in this length group as metadata. This creates independent work packages that can be distributed efficiently by OpenMP to the available processor cores without any synchronization effort.

```
1     std::vector<size_t> q_indices(num_queries);
2     std::iota(q_indices.begin(), q_indices.end(), 0);
3
4     std::sort(q_indices.begin(), q_indices.end(), [&](size_t a, size_t b) {
5         return queries[a].length() < queries[b].length();
6     });
7
8     std::vector<LengthBlock> blocks;
9     for (size_t i = 0; i < num_queries; ) {
10        size_t len = queries[q_indices[i]].length();
11        size_t start = i;
12        while (i < num_queries && queries[q_indices[i]].length() == len) {
13            i++;
14        }
```

```
15          if (len > 0 && len <= text_length) {
16              blocks.push_back({len, start, i - start});
17          }
18      }
```

Listing 5: New Data Struct

4.3 Reference Implementation

Finally, we also parallelized the reference implementation by distributing the queries across the threads. In the `std_openmp` implementation, using OpenMP, this modification required adding only a single line of code to the already very simple and concise reference implementation.

5 Using the GPU

5.1 OpenMPI

6 Summary

References

[1] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987. DOI: 10.1147/rd.312.0249

[2] Free Software Foundation, *The GNU C++ Library*, version 15.2.1, Files: libstdc++-v3/include/bits/basic_string.h, libstdc++-v3/include/bits/basic_string.tcc, libstdc++-v3/include/bits/char_traits.h. [Online]. Available: https://gcc.gnu.org/git/?p=gcc.git;a=tree